

GUÍA INTEGRAL DE SUSTENTACIÓN ORAL Y DEFENSA TÉCNICA

Proyecto de Fin de Carrera: Sistema de Gestión Académica (SGA) Distribuido
Institución: Escuela de Educación Básica "Provincias Unidas" · Período 2026-2027
Equipo PFC (BCEL): Bedon · **Castro (Leonardo - Líder sga-principal)** · Emanuel · Luna
Docente Evaluador: Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Componente	Detalle de Producción en AWS	Responsabilidad / Rol
sga-principal	Java 17 Spring Boot 3 · 4 Capas Limpias · Puerto 8080 y 9092	Leonardo Castro (Backend Central)
microservicio-docente	Python Django REST + gRPC Client · Puerto 8081 / 8000	Gestión de Cátedra y Asistencias
microservicio-ia	FastAPI + Google Gemini 1.5 Flash API · Puerto 8084	Tutoría Pedagógica Predictiva
sga-haproxy	HAProxy 2.9 (Capa 7 L7 LB) · Puertos 8080, 8081, 8084, 9092	API Gateway y Balanceo Distribuido
PostgreSQL 17	Master (5433) + Standby Réplica (5434) · WAL Streaming	Base de Datos Distribuida

1. Arquitectura Distribuida y Conexión de Microservicios

El sistema implementa una arquitectura de microservicios orientada al dominio y desacoplada mediante un **API Gateway L7 (HAProxy)** y un canal binario inter-procesos **gRPC (HTTP/2 con Protobuf)**:

- **Fuente de Verdad Central (sga-principal):** Aloja el catálogo institucional (grados, paralelos, materias, periodos) y gestiona la matrícula centralizada. Expone endpoints REST para administración y servicios gRPC para consumo de otros microservicios.
- **Consumo de Cátedra (microservicio-docente):** Cuando un docente ingresa a registrar calificaciones o asistencias en Django, el microservicio no duplica datos de alumnos en su base, sino que consulta directamente al servidor gRPC de sga-principal en el puerto 9092 en menos de 2 milisegundos.
- **Tutoría con IA (microservicio-ia):** Desarrollado en FastAPI, recibe el vector de notas del estudiante (7 formativas + 2 sumativas) y se conecta a la API oficial de **Google Gemini 1.5 Flash** para emitir un diagnóstico cualitativo automatizado.
- **Single Sign-On (SSO) y Handoff:** La autenticación genera tokens JWT firmados en sga-principal. Mediante el mecanismo de Handoff seguro, el usuario navega transparentemente entre el portal administrativo React y los microservicios sin reautenticarse.

■ LO QUE DICES ANTE EL INGENIERO (COMUNICACIÓN gRPC)

"Ingeniero, nuestro ecosistema no es un monolito. Está desacoplado en microservicios autónomos orquestados por HAProxy como API Gateway de Capa 7. Mi microservicio principal sga-principal (Spring Boot) es la fuente única de verdad para el catálogo institucional y matrícula. Para la comunicación inter-servicios de alta velocidad usamos gRPC con HTTP/2 y serialización binaria en Protocol Buffers en el puerto 9092."

2. Base de Datos Distribuida: Fragmentación y Partition Pruning

Para garantizar escalabilidad y tiempos de respuesta sub-milisegundo con altos volúmenes de concurrencia, aplicamos dos técnicas de fragmentación en PostgreSQL 17:

A. Range Partitioning en Asistencias (1.71 Millones de Filas)

La tabla sga_docente.asistencias está dividida en particiones trimestrales por rango de fechas (asistencias_2026_t1, t2, t3, default). Al consultar un rango de fechas, el planificador de PostgreSQL activa **Partition Pruning (Poda de Particiones)**: descarta los trimestres irrelevantes y escanea únicamente la partición requerida en O(1).

```
EXPLAIN ANALYZE
SELECT * FROM sga_docente.asistencias
WHERE fecha BETWEEN '2026-05-01' AND '2026-05-31';
-- Resultado: Bitmap Heap Scan on asistencias_2026_t2 (descarta t1 y t3). Tiempo: < 1.3 ms
```

B. Hash Sharding en Calificaciones (141K Filas en 4 Shards)

La tabla sga_docente.calificaciones implementa **Hash Sharding** con MODULUS 4 sobre id_matricula, distribuyendo uniformemente las calificaciones entre 4 shards físicos (calificaciones_shard_0 a shard_3) para evitar saturación de I/O en escrituras concurrentes.

```
SELECT tableoid::regclass AS shard_activo, count(*) AS total_notas
FROM sga_docente.calificaciones
GROUP BY tableoid;
-- Resultado: shard_0: 33,972 | shard_1: 38,438 | shard_2: 38,494 | shard_3: 30,583
```

C. Alta Disponibilidad y Replicación WAL Streaming

El clúster cuenta con replicación física en tiempo real mediante **Write-Ahead Logging (WAL) Streaming** entre el nodo Maestro (5433) y el nodo Réplica Standby (5434).

```
SELECT client_addr, application_name, state, sync_state,
       pg_wal_lsn_diff(pg_current_wal_lsn(), write_lsn) AS lag_bytes
FROM pg_stat_replication;
-- Resultado: walreceiver | state: streaming | sync_state: async | lag_bytes: 0
```

3. Arquitectura en 4 Capas y los 5 Patrones GoF de BCEL

En sga-principal implementamos Clean Architecture cumpliendo principios SOLID y separando responsabilidades en 4 paquetes estrictos:

Capa	Paquete Java	Responsabilidad Técnica y Principio SOLID
Presentation	ec.edu.uteq.sga.presentation	Controladores REST, DTOs de entrada y salida, manejo global de excepciones (Single Responsibility).
Application	ec.edu.uteq.sga.application	Servicios de negocio transaccionales (@Service), orquestación de casos de uso y lógica de dominio.
Domain	ec.edu.uteq.sga.domain	Entidades JPA de dominio puro, enumeraciones de estado y contratos sin dependencias de infraestructura.
Infraestructure	ec.edu.uteq.sga.infrastructure	Repositorios Spring Data JPA, adaptadores gRPC (@GrpcService), seguridad JWT y filtros CORS.

Los 5 Patrones de Diseño GoF Asignados a BCEL:

1. **Repository:** Abstracción desacoplada de acceso a base de datos mediante interfaces Spring Data (ej. EstudianteRepository).
2. **Strategy:** Esquemas intercambiables para validación y ponderación de notas ministeriales (70% formativa + 30% sumativa).
3. **Facade:** En PrincipalGrpcService, unificando consultas complejas a múltiples entidades en métodos gRPC simples de alto rendimiento.
4. **Observer:** En el sistema de eventos de auditoría y notificaciones automáticas al representante.
5. **Template Method:** En los flujos estructurados de consolidación y cierre de actas trimestrales.

4. Banco de Preguntas Típicas del Evaluador y Respuestas

P1: ¿Por qué usaron gRPC en lugar de REST para comunicar microservicios?

"Ingeniero, REST usa JSON en texto plano sobre HTTP/1.1, lo que produce sobrecarga por parsing y headers repetitivos. gRPC utiliza Protocol Buffers en formato binario y opera sobre HTTP/2 multiplexado en un solo canal TCP. Esto reduce el payload en más de un 60% y permite latencias sub-2ms entre Spring Boot y Django en el puerto 9092."

P2: ¿Cómo demostró que la poda de particiones (Partition Pruning) realmente funciona?

"Lo demostramos ejecutando `EXPLAIN ANALYZE SELECT * FROM sga_docente.asistencias WHERE fecha BETWEEN ...`. El optimizador descarta en tiempo de planificación las particiones de los demás trimestres y ejecuta un Bitmap Heap Scan exclusivamente sobre `asistencias_2026_t2`, reduciendo el tiempo a 0.8 ms en 1.7 millones de filas."

P3: ¿Qué ocurre si el nodo maestro de PostgreSQL se cae?

"El clúster cuenta con un nodo Standby Réplica sincronizado en streaming WAL constante con `lag_bytes = 0`. Ante una indisponibilidad del nodo 5433, el balanceador o administrador ejecuta `pg_ctl promote` en el nodo 5434 para asumir el rol primario sin pérdida transaccional ($RPO \approx 0$)."

P4: ¿Cómo protege la privacidad del estudiante el microservicio de IA?

"El microservicio `microservicio-ia` implementa minimización de datos: solo envía el vector numérico de 9 notas y el nombre de la asignatura a Google Gemini 1.5 Flash. No se transmiten cédulas, direcciones ni información sensible a la API externa. Además cuenta con fallback heurístico local en caso de caída de internet."

P5: ¿Por qué estructuraron sga-principal en 4 capas?

"Para cumplir con Clean Architecture y el principio de Inversión de Dependencias (DIP). El dominio no depende de la base de datos ni de frameworks externos. La capa de presentación solo maneja HTTP/REST, la de aplicación orquesta la lógica, el dominio contiene las reglas de negocio puras y la infraestructura encapsula JPA, gRPC y seguridad."